

CS3780/5780 Prelim Study Guide

Practice problems from past exams

Instructions: This guide has three problems from past CS3780/5780 exams for each prelim topic. Solutions follow each topic.

Problem 1: k-Nearest Neighbors

1. True or False: In the limit (as the number of data points $n \rightarrow \infty$) the error of the 1-NN classifier is at most twice the error of the Bayes Optimal classifier.
2. [3 pts] Given a training dataset of n samples, d -dimensional data points, and Euclidean distance as the distance measure, what is the compute time required to classify a (single) new test point using k nearest neighbors, if you compute the distance from the test point to every training point and then sort all n distances?

Express your answer in $O(\cdot)$ notation and briefly explain how you arrived at this complexity. (2 sentences suffice.)

3. [20 pts] The scatter plot below represents a classification problem with two features, x_1 and x_2 , where points are labeled with two different classes (represented by red bullets and blue squares).



Choosing the right value of k in the nearest neighbor approach is important. One approach used for picking k is the following: For every k we are considering, we compute the so-called leave-one-out error over training data as follows:

- For each training point, pretend it is a test point and use the KNN rule (with Euclidean distance) to predict its label.
- If the prediction is wrong, count it as a mistake.
- Now, the leave-one-out error is the fraction of training points that are misclassified.

NOTE: In the following subquestions, if using k neighbors returns a tie between both classes, default to using $k - 1$ nearest neighbors.

- (a) [6 pts] For each of $k = 1, 2$, and 3 , calculate the average leave-one-out error over all $n = 8$ training samples.
- (b) [4 pts] Of the choices of $k \in \{1, 2, 3\}$ above, which is the best choice for k based on the computed leave-one-out error? In 1–2 sentences, reason why such a k is ideal. (Answers that only state choice by low leave-one-out error will not receive full credit.)

- (c) [5 pts] For the training data shown above, what is the training (not test) error using KNN with $k = 1$ and why? (1 sentence suffices; 1–3 sentences OK.) Would the training error for perceptron be the same or lower? In 1–2 sentences, explain why or why not.
- (d) [3 pts] Consider the application of spam email classification. Let each email be represented as a feature vector using the counts of its words. The feature vectors for three emails in a $d = 10$ dimensional space are as follows:

Email	Feature vector	Label
A: Congrats!! You have won a free vacation. Click here to claim your prize.	[1 1 1 1 1 1 0 0 0 0]	spam ($y = +1$)
B: Congrats on your big achievement!! We'd love to feature your work.	[1 0 0 0 0 0 1 1 1 0]	not-spam ($y = -1$)
C: Congratulations!! Click here to win exciting prizes and work goodies.	[1 1 0 0 1 0 0 0 1 0]	

Now, with the similarity metric between two feature vectors u, v as the scaled dot product,

$$d(u, v) = \frac{1}{d} \langle u, v \rangle = \frac{1}{d} \sum_{i=1}^d u_i v_i,$$

classify email-C (on the last row) using KNN with $k = 1$.

- (e) [2 pts] In the above subquestion, consider adding 1,000 new dimensions to our feature vector, whose values are assigned by tossing a biased coin with $P(x_i = 0) = 0.8$. Under scaled dot product metric and $K = 1$, what is the expected contribution of these new dimensions to the metric? What can you say about KNN in high-dimensional spaces under the given similarity metric? (1 sentence suffices; 1–3 sentences OK.)

Hint: Think about computing $E[u_i v_i]$ for two feature values, u_i and v_i , and using the linearity of expectation.

Solutions

1. True

As $n \rightarrow \infty$ the nearest neighbor lies arbitrarily close to the test point, so 1-NN errs only when the two labels disagree, which happens with probability $2\epsilon^*(1 - \epsilon^*) \leq 2\epsilon^*$, where ϵ^* is the Bayes error.

Source: Spring 2023 Prelim, Q2.6

2. Computing the distance to one training point takes $O(d)$, so all n distances take $O(nd)$. Sorting them takes $O(n \log n)$, and taking a majority vote over the first k takes $O(k)$, for a total of $O(nd + n \log n)$ (since $k \leq n$).

(Without sorting, selecting the k smallest distances one at a time gives $O(nd + kn)$, which is also acceptable if explained.)

Source: Spring 2025 Prelim, Q1(c)

3. (a) For $k = 1$, the three points in the left-most “triangle” (two blue squares and the red bullet) would be misclassified. Hence, the average leave-one-out error for $k = 1$ is $3/8$.
For $k = 2$, with the $k - 1$ tie-breaking strategy, we would still have the same three points in the left-most triangle misclassified. Hence, the average leave-one-out error for $k = 2$ is also $3/8$.
For $k = 3$, only one point (= the red bullet within the left blue square quad) would be misclassified. Hence, the average leave-one-out error for $k = 3$ is $1/8$.
- (b) For $k = 3$, the leave-one-out error was the lowest ($= 1/8$), hence optimal choice for k is 3. Possible justifications for $k = 3$ include reasoning about the layout of the points, or by explaining why $k = 1, 2$ are worse, because the outlier red point causes the leftmost blue points to be misclassified as well.

- (c) The training error in KNN with $k = 1$ is 0 since each point is its own closest point; hence, we “know” the associated label. No, the training error for perceptron would not be the same (or lower), because the given dataset is not linearly separable. Hence, the training error would be greater than 0.
- (d) Using the scaled dot product, $d(C, A) = 3/10$ and $d(C, B) = 2/10$. Using KNN with $k = 1$, email-C would be (correctly) classified as being spam ($y = +1$).
- (e) We realize that the scaled dot product between two vectors, upon adding new dimensions can be realized as

$$d(u, v) = \frac{1}{d+1000} \sum_{i=1}^{d+1000} u_i v_i = \frac{1}{d+1000} \left(\sum_{i=1}^d u_i v_i + \sum_{l=d+1}^{d+1000} u_l v_l \right).$$

Noting that u_i and v_i are drawn IID from Bernoulli with $p = 0.2$, the expected value of each $u_i v_i$ is

$$E[u_i v_i] = E[u_i] E[v_i] = P(u_i = 1)P(v_i = 1) = 0.2 \times 0.2 = 0.04.$$

From the linearity of expectation, the expected contribution of the newly added 1,000 dimensions (ignoring $1/(d+1000)$ constant) is

$$E \left[\sum_{l=d+1}^{d+1000} u_l v_l \right] = \sum_{l=d+1}^{d+1000} E[u_l v_l] = 1000 \times 0.04 = 40.$$

Hence, under scaled dot-product, in expectation, $d(C, A) = (3 + 40)/1010$ and $d(C, B) = (2 + 40)/1010$, which are both quite similar. Hence, in high-dimensional spaces, the notion of “closeness” breaks down, rendering KNN ineffective.

Source: Spring 2026 Practice Prelim, Q1

Problem 2: Perceptron

1. [2 pts] What condition(s) are necessary for the Perceptron algorithm to converge? Assuming it does converge, what property of the training dataset affects how long the Perceptron algorithm runs?

2. [9 pts]

- (a) [5 points] Your friend has a linearly separable dataset (with a good margin) and intends to run the perceptron algorithm on it. However, their implementation contains two modifications:

- i. Instead of updating the weight vector only when a mistake occurs ($y_j w^T x_j \leq 0$), they mistakenly set the update condition to trigger when $y_j w^T x_j \geq 0$.
- ii. Additionally, the update rule itself is incorrect: instead of the correct update $w \leftarrow w + y_j x_j$, they incorrectly implemented it as $w \leftarrow w - y_j x_j$, subtracting instead of adding.

Will your friend's algorithm converge on this linearly separable dataset? If "yes," why would you expect it to converge? If "no," what prevents convergence? Explain in under 4 sentences.

- (b) [2 points] Assume that the modified perceptron algorithm (presented in the previous subquestion) has been run for a significant number of weight updates, resulting in a final weight vector.

Given this weight vector, is there a way to obtain a good classifier on the data without restarting the perceptron algorithm from scratch? If so, how might you use this weight vector to construct a classifier with low training error? Justify; 1 sentence OK, 1–3 sentences acceptable.

- (c) [2 points] Now, you corrected your friend's code and ran the perceptron algorithm to convergence on the linearly separable training dataset and used the resulting model to classify the test dataset. Despite the training data being linearly separable and the algorithm converging, the test error is not zero.

Should this raise concerns about an error in your perceptron code? If not, how can you explain the non-zero test error? (2 sentences suffice.)

3. [6 pts] Consider the following two training data samples S_1 and S_2 with $\vec{x}_i \in \mathbb{R}^2$ and $y \in \{+1, -1\}$.

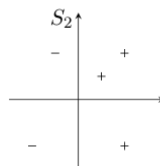
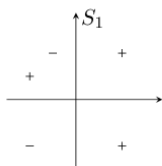
Circle the correct answer for the following multiple choice questions. No explanation needed.

- (a) The Perceptron algorithm will fail to terminate in finite time for

- i. S_1 , but not S_2
- ii. S_2 , but not S_1
- iii. both S_1 and S_2
- iv. neither S_1 nor S_2

- (b) Logistic Regression (unregularized) will run into numerical overflow errors for

- i. S_1 , but not S_2
- ii. S_2 , but not S_1
- iii. both S_1 and S_2
- iv. neither S_1 nor S_2



Solutions

1. The Perceptron algorithm converges if the dataset is linearly separable. Its runtime can depend on the maximum margin of a separating hyperplane, once the dataset is normalized. (No points lost for not mentioning normalization.)

Source: Spring 2022 Final, Q2.3

2. (a) With the two modifications above, the “incorrect” algorithm is essentially moving w away from x_j every time the classifier gets the point x_j correct.

The above is equivalent to running the standard perceptron with all the labels flipped, i.e., all positive-labeled points now become negative-labeled and vice versa. Hence, the resultant weight vector here would be the same as that in perceptron, but in the opposite direction. Now, since the separating hyperplane is perpendicular to w , it is also perpendicular to $-w$. In other words, we realize the same decision boundary under the two “mistakes” noted in the problem. (Aside: The phrase, “two wrongs make a right!” comes to mind.)

Consequently, yes, the algorithm converges on a linearly-separable dataset.

- (b) From the observations above, we realize that the resultant w from the modified perceptron algorithm is just in the opposite direction to that of the original perceptron algorithm (say $w_{\text{perceptron}}$). Hence, $w_{\text{perceptron}} = -w$.
- (c) The key distinction here is that simply because *training* error is zero doesn’t automatically imply test error (computed on previously unseen data) is also zero.

Hence, we need not worry, so long as the test error is not really large, in which case, our IID assumption might be broken.

Source: Spring 2025 Prelim, Q1(d)-(f)

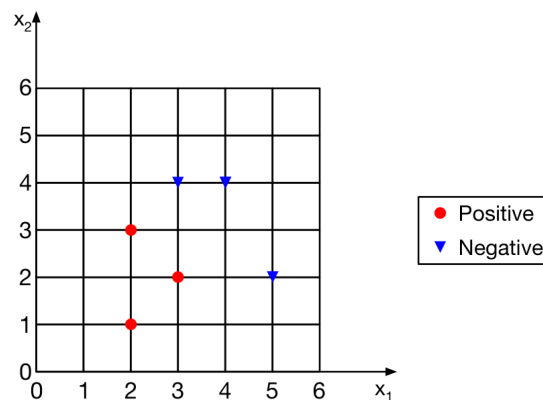
3. (a) S_1 only (because it is not linearly separable)
(b) S_2 only (because it is linearly separable, so the optimal $\|\vec{w}\| \rightarrow \infty$)

Source: Fall 2025 Prelim, Q4.1

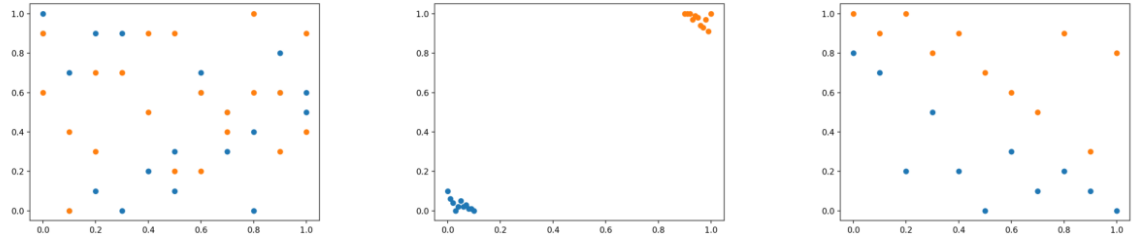
Problem 3: Support Vector Machines

1. [2 pts] True/False: If $0 \leq \xi_i < 1$ for all training examples, then the SVM classifier must achieve zero training error. Include a one sentence explanation.
2. [14 pts] We want to apply support vector machine (SVM) to solve a binary classification problem for the following data set. We present the data set in both table and figure for your convenience. In the figure, red circles are positive data points and blue triangles are negative data points.

x_1	x_2	y
2	1	+1
2	3	+1
3	2	+1
3	4	-1
4	4	-1
5	2	-1

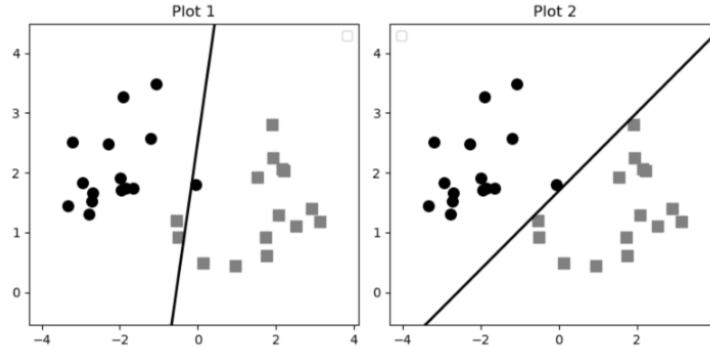


- (a) (8 pts) Let's say we have a hard margin SVM.
 - (i) (4 pts) List all support vectors, and write down the algebraic form of the hyperplane with maximum margin. No justification required.
 - (ii) (4 pts) For each of the following data points (x, y) , if we add it to the current training set, will the solution of the hard margin SVM change (no justification required)?
 - (yes/no) $((1, 5), +1)$
 - (yes/no) $((1, 1), +1)$
 - (yes/no) $((4, 1), -1)$
 - (yes/no) $((3, 5), -1)$
 - (iii) (3 pts) What is the main limitation of hard margin SVMs that soft margin SVMs overcome? Hint: the limitation is a property of the data set to which we apply the SVMs.
- (b) (3 pts) Another example of a linear classifier is Perceptron. Rank the following data sets based off of the expected number of update steps Perceptron will make from the fewest to the most (left, middle, right below).



3. [5 pts]

- For the SVMs below (which both use the same dataset), which one is more likely to correspond to $C = 0.01$, and which one to $C=100$? Include a brief explanation.
- Recall that in soft-margin SVMs, $\sum \xi_i$ provides an upper bound on the number of training errors. This bound can be tighter (closer to the actual number of errors) or looser (much larger than the actual number of errors). Between $C = 0.01$ or $C = 100$, which value of C is more likely to yield a tighter bound **relative to its own training errors**?



Solutions

1. **True**

Since $y_i(w^\top x_i + b) \geq 1 - \xi_i > 0$ for every i , every training point lies on the correct side of the hyperplane.

Source: Fall 2025 Prelim, Q5.1

- (2 pts) Support vectors: $((2, 3), +1)$, $((3, 2), +1)$, $((3, 4), -1)$, $((5, 2), -1)$. (2 pts) Hyperplane: $x_1 + x_2 - 6 = 0$
 - yes, no, yes, no
 - Hard margin SVM only applies to *linearly separable* data sets, i.e. data for which a *separating hyper-plane* can be found. Soft margin SVMs can be applied to data sets for which no such hyper-plane exists.
- middle, right, left.

We expect Perceptron to converge the fastest on the middle data set because the points in the different classes are very far apart. The dataset on the right is linearly separable, however the classes are much closer together, so it is harder to find a valid separating plane than in the middle example.

We expect the dataset on the left to take the most amount of time because the classes are not actually linearly separable so no valid separating plane can be found.

Source: Fall 2021 Prelim, Q6

- Plot 1 has $C = 0.01$ since it misclassified more points than plot 2.

- (b) $C = 100$ produces the best bound in this case since it penalizes slack variables, and they are likely to be smaller even for examples that are misclassified. Hence, $C = 100$ is likely to produce the best bound.

Source: Fall 2025 Prelim, Q5.2

Problem 4: Linear Classification

1. [3 pts] Suppose that you are using Logistic Regression for a binary classification problem, $\forall i, y_i \in \{+1, -1\}$. The model has the following parameters:

$$w = [2 \quad 1 \quad -1]^\top \quad b = -1$$

Consider a point x_t such that:

$$x_t = [1 \quad -1 \quad 0]^\top$$

Calculate the probability $P(y_t = +1|x_t)$ and $P(y_t = -1|x_t)$.

2. [8 pts] You have a binary classification problem with tens of thousands of features. You model the data with the likelihood model $Y_i \sim \text{Bernoulli}(\sigma(\vec{w} \cdot \vec{x}_i))$, i.e. $P(Y_i = y_i | \vec{x}_i, \vec{w}) = \frac{1}{1 + \exp(-y_i \vec{w} \cdot \vec{x}_i)}$. You know that only a small number of features are important. In this question, we will model this knowledge using a prior to encode that the weight vector does not have very many nonzero entries. Recall the L_0 norm $\|\vec{w}\|_0 = \sum_{j=1}^d \mathbf{1}\{|w_j| > 0\}$. We will use the prior $P(W = \vec{w}) = \frac{1}{z} \exp(-\frac{1}{b} \|\vec{w}\|_0)$ where z is a normalization constant to ensure that this is a valid probability density.

Use Maximum A Posteriori to derive a regularized linear learning objective of the form

$$\min_{\vec{w}} R(\vec{w}) + C \sum_{i=1}^n L(\vec{w} \cdot x_i, y_i).$$

Show the steps of your derivation.

3. [10 pts] We have learned the binary logistic regression model in the class, but in the real world we usually have more than 2 labels in a dataset. In this scenario, we would have a **multiclass logistic regression** problem as follows:

Given a dataset with k labels as $\{L_1, L_2, \dots, L_k\}$ and a set of k weight vectors as $\{w_1, w_2, \dots, w_k\}$, we would have

$$P_{\text{multiLR}}(y = L_i | x) = \frac{\exp(w_i^\top x)}{\sum_{j=1}^k \exp(w_j^\top x)}$$

- (a) (4 points) Assume you have trained multiclass classification for $k = 2$ classes. Given a vector v , remember that for *binary* logistic regression the conditional label distribution is defined as:

$$P_{\text{binLR}}(y = +1 | x) = \frac{1}{1 + \exp(-v^\top x)}$$

$$P_{\text{binLR}}(y = -1 | x) = \frac{1}{1 + \exp(v^\top x)}$$

Assume you want to convert your multi-class classifier to the binary setting. Let label L_1 correspond to $+1$ and L_2 to -1 . Given your vectors w_1, w_2 , derive a vector v such that the resulting binary classifier matches your results of multi-class logistic regression.

- (b) (2 points) Name one advantage that multi-class LR has over the k -NN classifier, and one advantage vice versa.
- (c) (4 points) Consider a dataset $D = \{(0, -1), (4, +1), (5, +1)\}$ consisting of data points (x_i, y_i) with 1-dimensional features x_i and binary labels $y_i \in \{-1, +1\}$.

We now apply logistic regression on the dataset D . Similar to what we have seen in class, we represent the features as a 2-dimensional vectors $x_i = \begin{bmatrix} x_i \\ 1 \end{bmatrix}$. Logistic regression then returns a

weight vector $w = \begin{bmatrix} 10 \\ -20 \end{bmatrix}$. Show that this logistic regression model is identical to a 1-NN classifier, i.e. show that for any test point with 1-dimensional feature x_t , the logistic regression model gives the same classification as a 1-NN classifier.

Solutions

1. $P(y_t = y|x_t) = \frac{1}{1+e^{-y(w^T x_t + b)}}$, and $w^T x_t + b = 1 - 1 = 0$.

$P(y_t = +1|x_t) = \frac{1}{1+e^{-1(0)}} = 1/2$. Similarly, $P(y_t = -1|x_t) = 1/2$.

Source: Spring 2023 Final, Q3.5

2.

$$\begin{aligned}\hat{w} &= \operatorname{argmax}_{\vec{w}} P(y_1, \dots, y_n, \vec{w} | x_1, \dots, x_n) \\ &= \operatorname{argmax}_{\vec{w}} p(\vec{w}) \prod_{i=1}^n P(y_i | x_i, \vec{w}) \\ &= \operatorname{argmax}_{\vec{w}} \log P(\vec{w}) + \sum_{i=1}^n \log P(y_i | x_i, \vec{w}) \\ &= \operatorname{argmin}_{\vec{w}} -\log P(\vec{w}) - \sum_{i=1}^n \log P(y_i | x_i, \vec{w}) \\ &= \operatorname{argmin}_{\vec{w}} \frac{1}{b} \|\vec{w}\|_0 + \log z + \sum_{i=1}^n \log(1 + \exp(-y_i \vec{w} \cdot \vec{x}_i)) \\ &= \operatorname{argmin}_{\vec{w}} \|\vec{w}\|_0 + b \sum_{i=1}^n \log(1 + \exp(-y_i \vec{w} \cdot \vec{x}_i)).\end{aligned}$$

So $R(\vec{w}) = \|\vec{w}\|_0$, $L(\vec{w} \cdot \vec{x}_i, y_i) = \log(1 + \exp(-y_i \vec{w} \cdot \vec{x}_i))$ (the logistic loss), and $C = b$.

Source: Fall 2025 Prelim, Q4.3

3. (a)

$$P(y = +1 | x) = \frac{\exp(w_1^\top x)}{\exp(w_1^\top x) + \exp(w_2^\top x)}$$

Divide the numerator and denominator by $\exp(w_1^\top x)$

$$P(y = +1 | x) = \frac{1}{1 + \exp((w_2 - w_1)^\top x)}$$

Similarly, solving for $P(y = -1 | x)$ from the Multiclass expression, we get

$$P(y = -1 | x) = \frac{1}{1 + \exp((w_1 - w_2)^\top x)}$$

Compare this with the binary logistic regression expression

$$P(y = -1 | x) = \frac{1}{1 + \exp(v^\top x)} = \frac{1}{1 + \exp((w_1 - w_2)^\top x)}$$

Simplifying this gives us $v = w_1 - w_2$.

(b) Advantage of LR over kNN: Logistic Regression works in high dimensions and does not assume each feature to be weighted equally.

Advantage of kNN over LR: kNN is useful when we are dealing with non-linear decision boundaries, whereas LR assumes linear decision boundaries.

(c) The augmented test point is $\begin{bmatrix} x_t \\ 1 \end{bmatrix}$.

1-NN: the training points are 0 (label -1), and 4 and 5 (label +1). The nearest positive point is always at least as close as 5 is, so 1-NN predicts -1 iff $|x_t - 0| < |x_t - 4|$, i.e. iff $x_t < 2$, and +1 iff $x_t > 2$ ($x_t = 2$ is a tie).

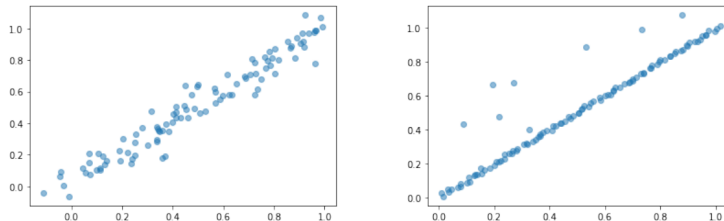
Logistic regression: $w^\top x_t = 10x_t - 20 = 10(x_t - 2)$, so $P(y = +1 \mid x_t) = \frac{1}{1 + \exp(-10(x_t - 2))}$, which is $> 1/2$ iff $x_t > 2$ and $< 1/2$ iff $x_t < 2$ (exactly $1/2$ at $x_t = 2$).

Hence both classifiers predict -1 for $x_t < 2$ and $+1$ for $x_t > 2$: they are identical.

Source: Spring 2023 Prelim, Q6

Problem 5: Linear Regression

1. [4 pts] You get two regression datasets, shown below: dataset A (left) and dataset B (right). Say you are required to use L1 loss $\ell_1(y, \hat{y}) = |\hat{y} - y|$ for one of them and L2 loss $\ell_2(y, \hat{y}) = (\hat{y} - y)^2$ for the other. State which loss you would choose for each dataset and explain why.



2. [3 pts] You are training a classification model, but realize that most of the features might not be important, and want to encourage sparsity in the weights of your model. Which regularizer(s) can induce sparsity (for appropriately set values of $\lambda > 0$, $\mu > 0$)? Select all that apply.

- $\lambda \|w\|_1$
- $\mu \|w\|_2^2$
- $\lambda \|w\|_1 + \mu \|w\|_2^2$

3. [5 pts] In class, we saw that the standard Ridge Regression objective function for a dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ is:

$$L(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

where $\mathbf{w} \in \mathbb{R}^d$, $\mathbf{x}_i \in \mathbb{R}^d$, and $\lambda > 0$ is a single regularization scalar applied uniformly to all weights.

In this problem, we will explore a “Generalized” Ridge Regression model. Your friend, who is an aspiring data scientist, decides that they have different prior beliefs about the importance of each feature. They propose a new objective function that penalizes each weight w_j individually like so:

$$L_G(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \frac{1}{2} \sum_{j=1}^d \lambda_j w_j^2$$

Here, $\lambda_j > 0$ is a **separate** regularization strength for each feature j .

This can be written in matrix form as:

$$L_G(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{1}{2} \mathbf{w}^\top \mathbf{\Lambda} \mathbf{w}$$

where $\mathbf{X} \in \mathbb{R}^{n \times d}$, $\mathbf{y} \in \mathbb{R}^n$, and $\mathbf{\Lambda}$ is a $d \times d$ diagonal matrix:

$$\mathbf{\Lambda} = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_d \end{bmatrix}$$

Consider the following 2D dataset ($d = 2$) with $n = 3$ data points:

x_1	x_2	y
1	0	2
0	1	0
-1	0	-2

Compute the generalized ridge regression solution $\hat{\mathbf{w}}(\lambda_1, \lambda_2)$ in terms of λ_1 and λ_2 . Show your work, including the $\mathbf{X}$ and $\mathbf{y}$ matrices, $\mathbf{X}^\top \mathbf{X}$, and $\mathbf{X}^\top \mathbf{y}$.

(Hint: The inverse of a 2x2 diagonal matrix is: $\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}^{-1} = \begin{bmatrix} 1/a & 0 \\ 0 & 1/b \end{bmatrix}$)

Solutions

1. Dataset B (right): L1 loss. It lies almost exactly on a line except for a few large outliers, and the squared loss would let those outliers pull the fit away; L1 is more robust. Dataset A (left): L2 loss. Its noise is spread evenly around the line with no outliers, so squared loss (the Gaussian-noise MLE) is appropriate.

Source: Fall 2022 Final, Q5.2

2. $\lambda \|w\|_1, \lambda \|w\|_1 + \mu \|w\|_2^2$

Source: Spring 2024 Final, Q2.3

3. First, form the matrices $\mathbf{X}$, $\mathbf{y}$, and $\mathbf{\Lambda}$:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & 0 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 0 \\ -2 \end{bmatrix}, \quad \mathbf{\Lambda} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

Next, compute the required components $\mathbf{X}^\top \mathbf{X}$ and $\mathbf{X}^\top \mathbf{y}$:

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} (1 \cdot 1 + 0 \cdot 0 + -1 \cdot -1) & (1 \cdot 0 + 0 \cdot 1 + -1 \cdot 0) \\ (0 \cdot 1 + 1 \cdot 0 + 0 \cdot -1) & (0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0) \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ -2 \end{bmatrix} = \begin{bmatrix} (1 \cdot 2 + 0 \cdot 0 + -1 \cdot -2) \\ (0 \cdot 2 + 1 \cdot 0 + 0 \cdot -2) \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \end{bmatrix}$$

Setting the gradient $\nabla L_G(\mathbf{w}) = -\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \mathbf{\Lambda}\mathbf{w}$ to zero gives $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \mathbf{\Lambda})^{-1} \mathbf{X}^\top \mathbf{y}$

$$(\mathbf{X}^\top \mathbf{X} + \mathbf{\Lambda}) = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} = \begin{bmatrix} 2 + \lambda_1 & 0 \\ 0 & 1 + \lambda_2 \end{bmatrix}$$

The inverse of this diagonal matrix is:

$$(\mathbf{X}^\top \mathbf{X} + \mathbf{\Lambda})^{-1} = \begin{bmatrix} \frac{1}{2 + \lambda_1} & 0 \\ 0 & \frac{1}{1 + \lambda_2} \end{bmatrix}$$

Finally, compute $\hat{\mathbf{w}}$:

$$\hat{\mathbf{w}} = \begin{bmatrix} \frac{1}{2 + \lambda_1} & 0 \\ 0 & \frac{1}{1 + \lambda_2} \end{bmatrix} \begin{bmatrix} 4 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{4}{2 + \lambda_1} \\ 0 \end{bmatrix}$$

Source: Fall 2025 Final (Alternate), Q7.2

Problem 6: ML Basics

1. It is always safe to split your data uniformly at random into training/validation/test set.
2. In a typical train/validation/test setup, we are first given a training dataset D_{train} , we then select a subset of this training dataset and call it our validation set $D_{\text{validation}}$, and we then further subselect part of the validation set and call it the test set D_{test} , such that $D_{\text{test}} \subset D_{\text{validation}} \subset D_{\text{train}}$.
3. [5 pts] A colleague claims they have built a medical image classifier that achieves 10% test error, far better than the 25% error of previous methods. You ask about their evaluation procedure: They had 2000 labeled images, kept 400 aside as a test set, and then trained 100 different classifiers on the remaining 1600 images. After checking all 100 models on the 400 test images, they reported the lowest observed error (10%). They did not use a validation set. Based on this methodology, do you trust that the true generalization error of the chosen classifier is close to 10%? Why or why not?

Solutions

1. **False**

Not if e.g. there is a significant temporal component (e.g. ham vs. spam data).

Source: Spring 2023 Prelim, Q2.10

2. **False**

The train, test, and validation datasets must be disjoint, and are selected via a partition of a single large dataset.

Source: Spring 2022 Prelim, Q1.2

3. No. Because the test set was repeatedly used for model selection, the reported 10% is an optimistically biased estimate. This procedure overfits to the test set just like training error overfits to the training set. Without a validation set, there is no unbiased estimate of the generalization error.

Source: Fall 2025 Prelim, Q3.2

Problem 7: Optimization

1. The loss of linear regression is convex while the loss of logistic regression is not convex.
2. [15 pts] When we learned about linear regression we talked about how using MAP estimation with a specific prior on w led to a formulation we called Ridge Regression—where a regularization term prevents w from becoming too large. Here, we are going to see how using gradient descent to solve the linear regression problem can have a naturally regularizing effect if we start with $w_0 = 0$ and cap the number of iterations.¹ For the sake of this question, we will have n data points $x_i \in \mathbb{R}^d$ with values $y_i \in \mathbb{R}$. Let

$$X = \begin{bmatrix} | & & | \\ x_1 & \dots & x_n \\ | & & | \end{bmatrix} \quad \text{and} \quad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

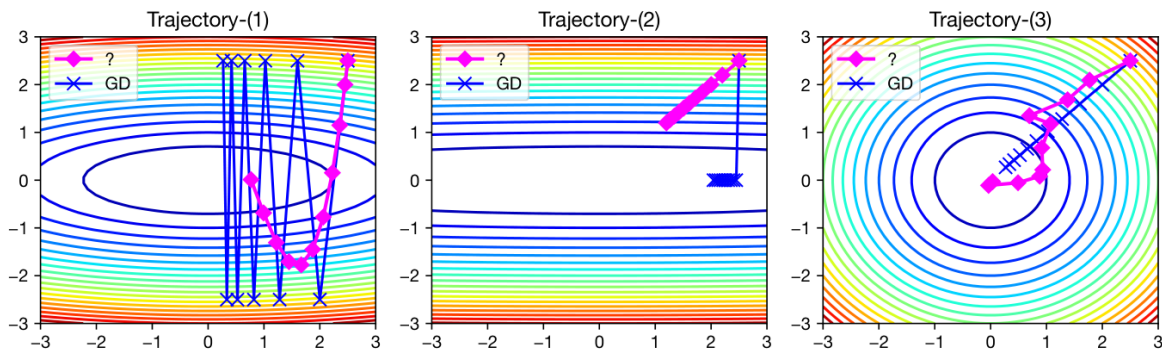
such that the linear regression problem may be phrased as $\min_w \frac{1}{2} \|X^\top w - y\|_2^2$.

- (a) (3 pts) Derive the gradient descent update (with step size α) for w_{t+1} given w_t .
 - (b) (3 pts) If we assume that α is small enough such that $\|(I - \alpha XX^\top)z\|_2 \leq \|z\|_2$ for any vector z then we can show that $\|w_k\|_2 \leq \alpha k \|Xy\|_2$ when $w_0 = 0$. Why does this suggest a potential regularizing effect if we stop gradient descent after a fixed number of iterations?
 - (c) (3 pts) If we run gradient descent for k steps are we guaranteed to find the best possible w satisfying $\|w\|_2 \leq \alpha k \|Xy\|_2$? Why or Why not?
 - (d) (5 pts) Now, still assuming that α is small enough such that $\|(I - \alpha XX^\top)z\|_2 \leq \|z\|_2$ for any vector z , show that $\|w_k\|_2 \leq \alpha k \|Xy\|_2$ when $w_0 = 0$. (Hint: remember the triangle inequality $\|a\| \leq \|b\| + \|c\|$ if $a = b + c$.)
3. [6 pts]

In class, we saw several variants of gradient descent, namely, momentum, AdaGrad (adaptive gradients), and stochastic gradient descent.

Below, we show the trajectories of optimizers chosen optimally for the underlying loss landscape depicted using contour lines. Identify (with justification) which optimizer might have resulted in the observed trajectory in each case. (1 sentence suffices for justification.)

Hint: Contrast the trajectory of the variant with that of vanilla gradient descent (in blue).



¹In reality, there is a bit more to it than what we are going to prove in this problem—you can write w_k as a filtered version of w^* , the optimal solution, that shows how quickly the solution converges in different eigenmodes.

Solutions

1. False

Logistic regression is also convex.

Source: Spring 2023 Prelim, Q2.2

2. (a) $w_{t+1} = w_t - \alpha g(w_t)$ where $g(w_t) = \nabla \left(\frac{1}{2} \|X^\top w_t - y\|_2^2 \right) = \frac{1}{2} \cdot 2X(X^\top w_t - y) = X(X^\top w_t - y) = XX^\top w_t - Xy$

$$= w_t - \alpha(XX^\top w_t - Xy) = w_t - \alpha XX^\top w_t + \alpha Xy$$
- (b) Stopping after k steps limits us to solutions with bounded norm. Constraining $\|w\|_2 \leq r$ is equivalent to ridge regression for a suitable λ , so early stopping acts like an L2 regularizer.
- (c) No. The bound only says that w_k lies in the ball $\|w\|_2 \leq \alpha k \|Xy\|_2$. Gradient descent is not solving the constrained problem $\min_w \frac{1}{2} \|X^\top w - y\|_2^2$ s.t. $\|w\|_2 \leq \alpha k \|Xy\|_2$; w_k is simply where k steps from 0 toward the unconstrained minimizer end up, which in general is not the best point in that ball.
- (d) We will prove this by induction. Base case: $\|w_0\| = 0 = \alpha \cdot 0 \cdot \|Xy\|$. Using the update from (a), $w_k = (I - \alpha XX^\top)w_{k-1} + \alpha Xy$. Now, assuming that $\|w_{k-1}\| \leq \alpha(k-1)\|Xy\|$ we have that

$$\begin{aligned} \|w_k\| &= \|(I - \alpha XX^\top)w_{k-1} + \alpha Xy\| \\ &\leq \|(I - \alpha XX^\top)w_{k-1}\| + \alpha \|Xy\| \\ &\leq \|w_{k-1}\| + \alpha \|Xy\| \quad (\text{assumption on } \alpha) \\ &\leq \alpha(k-1)\|Xy\| + \alpha \|Xy\| \\ &= \alpha k \|Xy\| \end{aligned}$$

Source: Fall 2022 Prelim, Q5

3. The underlying optimizers that generated the corresponding trajectory are:

- Trajectory-(1) (GD + momentum): Clearly a case of momentum. Here, we can clearly observe a “speed-up” as we follow in the same consistent direction, which reduces drastically when gradient changes direction.
- Trajectory-(2) (AdaGrad): AdaGrad rescales each coordinate by its accumulated gradient magnitude, so it keeps making progress along the flat direction, where vanilla GD barely moves.
- Trajectory-(3) (SGD): The updates are noisier than gradient descent’s and do not always point toward the minimum; hence, stochastic gradient descent.

Source: Spring 2025 Prelim, Q4(a)

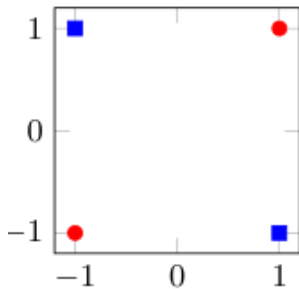
Problem 8: Kernel Methods

1. [3 pts] In class we discussed that the kernel-SVM classifier can be a “smart” nearest neighbor classifier. Which kernel function expressions make this true?

- linear kernel $k(x, z) = x^\top z$
- polynomial kernel $k(x, z) = (\gamma x^\top z + r)^d$
- RBF kernel $k(x, z) = \exp(-\gamma \|x - z\|^2)$

2. [5 pts] Consider the classic “XOR dataset” consisting of the four points $\{-1, 1\} \times \{-1, 1\}$ with label $y = x_1 x_2$, illustrated in the figure below.

Write out the kernel matrix (gram matrix) for this dataset for a polynomial kernel $k(x, z) = (1 + x^\top z)^d$ with degree $d = 1$ (you can order the data points in any order you want when writing out the matrix). Then, draw a plausible decision boundary for a hard-margin kernel SVM using this kernel—or if something would go wrong with using a hard-margin kernel SVM, explain what would happen and why. Would the behavior of the hard-margin kernel SVM on this dataset change if you used a larger d ? Explain.



3. [3 pts] Remember the rules for well-defined kernels:

- $k(x, z) = x^\top z$
- $k(x, z) = c k_1(x, z)$ for another kernel function k_1 for a constant $c \geq 0$
- $k(x, z) = k_1(x, z) + k_2(x, z)$ for kernel functions k_1 and k_2
- $k(x, z) = g(k_1(x, z))$ for another kernel function k_1 and a polynomial function g with positive coefficients
- $k(x, z) = k_1(x, z) k_2(x, z)$ for kernel functions k_1 and k_2
- $k(x, z) = e^{k_1(x, z)}$ for another kernel function k_1
- $k(x, z) = x^\top A z$ for a symmetric positive semidefinite matrix A

Using the rules for well-defined kernels (you can refer to rules (a)–(g)), prove that the following kernel is valid. Provide a brief justification.

$$k(x, z) = \exp((x^\top z)(x^\top z + 1)^3)$$

Solutions

1. The RBF kernel ($k(x, z) = \exp(-\gamma \|x - z\|^2)$) makes the kernel-SVM classifier function a “smart” nearest neighbor classifier because the exponential decay with distance assigns more weight to nearby points, resembling a “soft” nearest neighbor approach.

Source: Spring 2024 Final, Q3.3

2. With the order $(1, 1), (1, -1), (-1, 1), (-1, -1)$, the kernel matrix will have $k(x_i, x_i) = 3$ along the diagonal, then have the adjacent entries be 1 and the opposing entries be -1 . For example,

$$K = \begin{bmatrix} 3 & 1 & 1 & -1 \\ 1 & 3 & -1 & 1 \\ 1 & -1 & 3 & 1 \\ -1 & 1 & 1 & 3 \end{bmatrix}.$$

With $d = 1$ the feature map is $\phi(x) = [1, x_1, x_2]$, so the kernel SVM is just a linear classifier, and XOR is not linearly separable: the hard-margin constraints cannot all be satisfied, so the optimization problem is infeasible and there is no decision boundary. Yes, it changes for $d \geq 2$: the expansion of $(1 + x^\top z)^d$ contains the monomial $x_1 x_2$ as a feature, and $y = x_1 x_2$, so the data become linearly separable in feature space and the hard-margin SVM succeeds (boundary $x_1 x_2 = 0$, i.e. the two axes).

Source: Spring 2022 Final, Q3.2

3. Let $k_1 = x^\top z$ by rule (a). Let $k_2 = (k_1 + 1)^3 = (x^\top z + 1)^3$ by rule (d) where $g(x) = (x + 1)^3$. Let $k_3 = k_1 k_2 = (x^\top z)(x^\top z + 1)^3$ by rule (e). Finally, let $k_4 = \exp(k_3)$ by rule (f), which equals the kernel function above.

Source: Spring 2023 Final, Q6.2

Problem 9: Model Selection

1. If a classifier obtains 0% training error, it cannot have 100% testing error.
2. A classifier trained on less training data is less likely to overfit.
3. [4 pts] You are training a regression model using a polynomial kernel to predict housing prices. You have 1000 training examples and 200 validation examples. You train three models with different degree kernels and observe the following results:

Model	Training Error	Validation Error
Linear (degree 1)	15.2%	15.8%
Polynomial (degree 5)	8.3%	9.1%
Polynomial (degree 20)	0.1%	24.5%

Based solely on these results, which model would you select for deployment and why? In your answer, describe the error observed for each model in terms of underfitting and overfitting.

Solutions

1. **False**

Zero training error says nothing about unseen points. For example, 1-NN always has 0% training error, and it has 100% test error if every test point's nearest training neighbor has the opposite label.

Source: Fall 2021 Final, Q1.8

2. **False**

A specific classifier (with some fixed model complexity) will be more likely to overfit to noise in the training data when there is less training data, and is therefore more likely to overfit.

Source: Spring 2024 Final, Q1.2

3. Select the degree 5 polynomial model (middle option).

The linear model shows similar training and validation errors, suggesting it is underfitting - it doesn't have enough capacity to capture the underlying pattern in the data.

The degree 20 polynomial shows very low training error (0.1%) but much higher validation error (24.5%), indicating severe overfitting. The model has memorized the training data but fails to generalize to new examples.

The degree 5 polynomial achieves the best balance with moderate training error and the lowest validation error (9.1%). Since we care about performance on new, unseen data, validation error is the metric we should use for model selection. This model appears to have appropriate complexity for the problem.

Source: Fall 2025 Prelim, Q3.1